Materia: Programación IV

UNIDAD 8 · FORMULARIOS Y TABLAS

TRABAJO PRÁCTICO

CRUD de Categorías, Productos e Ingredientes (frontend) y de Pedidos (backend)

Revisor de la unidad: Prof. Franco González

Objetivos

Desarrollar una aplicación Full Stack conectando un frontend React + TypeScript con un backend FastAPI. El trabajo consiste en implementar el CRUD completo de Categorías, Productos e Ingredientes desde la interfaz de usuario, integrando el backend mediante TanStack Query, con formularios de TanStack Form, tablas de TanStack Table y estilos de Tailwind CSS. Los CRUD de Pedido y DetallePedido se construyen en el backend con la arquitectura Models-Schemas-Repository-UoW-Service-Router y se demuestran mediante archivos de REST Client.

Marco teórico

Concepto	Aplicación en el proyecto
React	Biblioteca para construir interfaces de usuario mediante componentes reutilizables.
TypeScript	Superset tipado de JavaScript para detectar errores en compilación y mejorar la legibilidad.
Vite	Servidor de desarrollo y herramienta de build. Con `pnpm create vite` se genera un proyecto React + TypeScript; es la alternativa recomendada a Create React App, que está discontinuado.
FastAPI	Framework Python de alto rendimiento para construir APIs REST. Permite definir endpoints, validar datos con Pydantic y exponer documentación automática en `/docs`.
Pydantic	Librería de validación de datos usada por FastAPI para definir modelos (schemas) a partir de anotaciones de tipos.
SQLModel	Biblioteca que define a la vez los modelos de tabla y los schemas a partir de clases de Python.

Concepto	Aplicación en el proyecto
Repository	Capa que encapsula el acceso a la base de datos de una entidad. No confirma la transacción.
Unit of Work	Capa que agrupa las operaciones de un caso de uso en una sola transacción y hace commit si todo sale bien o rollback si ocurre un error.
fetch nativo	API del navegador para realizar peticiones HTTP. Se usa con async/await dentro de las funciones `queryFn` y `mutationFn` de TanStack Query.
TanStack Query	Librería para obtener, cachear y sincronizar datos del servidor: `useQuery` lee y `useMutation` escribe, y tras cada mutación se invalida la consulta del listado con `invalidateQueries`.
React Router	Librería de enrutamiento: asocia cada ruta (`/categorias`, `/productos`, `/ingredientes`) con una página.
TanStack Form	Librería para formularios fuertemente tipados y escalables: `useForm` define los valores iniciales y el envío, `form.Field` conecta cada campo con su valor y sus errores, y los `validators` declaran las reglas de validación.
TanStack Table	Librería headless que gestiona la lógica de las tablas (columnas, ordenamiento, filtrado, paginación) sin imponer estilos; el marcado lo define el desarrollador con `flexRender`.
Interface	Construcción de TypeScript que define la forma de un objeto (por ejemplo, `Categoria` y `Producto`).
Tailwind CSS	Framework de CSS utilitario. Estilos aplicados directamente en el JSX mediante clases predefinidas.
CORS	Mecanismo del navegador por el cual el servidor indica qué orígenes pueden consultarlo. Es necesario porque el frontend (`http://localhost:5173`) y el backend (`http://localhost:8000`) usan puertos distintos y, por lo tanto, son orígenes distintos.

Consignas

Ejercicio 1 · Parte A — Backend con FastAPI

35 puntos

Tomando como base el trabajo práctico de la unidad 7: el backend ya tiene los CRUD de Categoría y Producto con PostgreSQL y SQLModel, y en esta unidad se agrega el módulo de Pedidos con la arquitectura de la Actividad 1.

a) Crear y activar un entorno virtual de Python, crear el archivo `requirements.txt` e instalar las dependencias:

```
python -m venv .venv
source .venv/bin/activate   # macOS/Linux
.venv\Scripts\activate      # Windows
pip install -r requirements.txt
```

b) Definir los models y schemas de Pedido y DetallePedido. El modelo simplifica el paquete Ventas & Transacciones del diagrama UML del trabajo práctico de la unidad 7: no se crean Usuario, DireccionEntrega, FormaPago, EstadoPedido ni HistorialEstadoPedido, y el estado del pedido se guarda como texto. Los campos mínimos son:

Pedido	DetallePedido
`id: int` (autoincremental)	`pedido_id: int` (FK a Pedido)
`fecha: datetime` (asignada por el backend)	`producto_id: int` (FK a Producto)
`estado: str` (PENDIENTE, CONFIRMADO, ENTREGADO o CANCELADO; al crear, PENDIENTE)	`cantidad: int` (mayor o igual a 1)
`total: float` (suma de cantidad por precio unitario de los detalles)	`precio_unitario: float` (el `precio_base` del producto al crear el pedido)
`deleted_at: datetime` (opcional; se completa en el borrado lógico)	

La clave primaria de DetallePedido es compuesta (`pedido_id` , `producto_id`), como en el diagrama UML.

c) Implementar el CRUD de Pedido y DetallePedido siguiendo la arquitectura `Models-Schemas-Repository-UoW-Service-Router` :

- **Repository:** encapsula las consultas de pedidos y detalles; no llama a `commit()` .
- **Unit of Work:** el del módulo `pedidos` abre la sesión, expone los repositorios que el caso de uso necesita (pedidos, detalles y productos) y, al salir del bloque `with` , hace commit si no hubo errores o rollback si se produjo una excepción.

- **Service:** aplica las reglas de negocio y ejecuta cada caso de uso dentro del Unit of Work.
- **Router:** `APIRouter(prefix="/pedidos", tags=["pedidos"])`; traduce las excepciones del servicio a respuestas HTTP (404 o 422).

Endpoints requeridos:

- `GET /pedidos` lista los pedidos no eliminados; `GET /pedidos/{id}` devuelve el pedido con sus detalles.
- `POST /pedidos` recibe una lista `detalles` (cada uno con `producto_id` y `cantidad`) y crea el pedido con sus detalles en una sola transacción (201). Un pedido sin detalles, con una cantidad menor que 1 o con un mismo `producto_id` repetido en dos detalles responde 422. Si un detalle referencia un producto inexistente, responde 404 y no se guarda nada.
- `PUT /pedidos/{id}` actualiza el estado; un valor fuera de los permitidos responde 422.
- `DELETE /pedidos/{id}` realiza un borrado lógico: completa `deleted_at` y responde 204. Desde ese momento, el pedido no aparece en el listado y operar sobre él responde 404.
- Todas las operaciones se demuestran con un archivo `pedidos.http` de REST Client, que incluye al menos un caso que responde 422 (por ejemplo, un producto repetido) y otro que responde 404.

El alta del pedido es el caso de uso que justifica el Unit of Work: si falla un detalle, el rollback descarta también el pedido.

(Código de ejemplo)

```
# app/modules/pedidos/service.py (fragmento)
class PedidoService:

    def __init__(self, uow):
        self.uow = uow

    def crear(self, data: PedidoCreate):
        with self.uow as uow:
            pedido = uow.pedidos.create(Pedido(estado="PENDIENTE", total=0))
            total = 0.0
            for item in data.detalles:
                producto = uow.productos.get_by_id(item.producto_id)
                if producto is None:
                    raise ProductoNoEncontrado(item.producto_id)
                uow.detalles.create(DetallePedido(
                    pedido_id=pedido.id,
                    producto_id=producto.id,
                    cantidad=item.cantidad,
                    precio_unitario=producto.precio_base,
                ))
                total += item.cantidad * producto.precio_base
            pedido.total = total
            return PedidoRead.model_validate(pedido)
```

Como en la Actividad 1, `create()` del repositorio agrega el objeto a la sesión y llama a `flush()`, por lo que `pedido.id` ya está disponible para los detalles. La excepción `ProductoNoEncontrado` provoca el rollback y el Router la traduce a una respuesta 404. La regla de producto no repetido se valida en el schema `PedidoCreate` o en el servicio, antes de crear los detalles: sin ella, la clave primaria compuesta rechaza el segundo detalle con un `IntegrityError` y el cliente recibe un error 500.

d) La solución integra lo trabajado en las unidades anteriores: los endpoints de Categoría y Producto del TP de la unidad 7 se mantienen, porque el frontend los consume, y el backend se prueba con archivos `.http` de REST Client. Para que la página de Ingredientes funcione, el backend expone además `GET /ingredientes`, `POST /ingredientes`, `PUT /ingredientes/{id}` y `DELETE /ingredientes/{id}`, con un modelo `Ingrediente` (`id: int`, `nombre: str`) y la misma estructura que los endpoints de Categoría del TP de la unidad 7.

e) Ejecutar el backend desde la carpeta raíz del proyecto:

```
python -m fastapi dev app/main.py
```

Notas

Se debe habilitar CORS en `app/main.py` (con `CORSMiddleware`) para que el frontend pueda consumir la API.

Estructura de ejemplo (la del proyecto de la Actividad 1, adaptada al dominio del TP). `core/repository.py` contiene el repositorio base con el CRUD genérico y `core/unit_of_work.py`, el Unit of Work base; el `unit_of_work.py` de cada módulo expone los repositorios que sus casos de uso necesitan. Los módulos de Categoría y Producto pueden quedar en las carpetas del TP de la unidad 7 (`app/categoria/` y `app/producto/`), y el de Ingrediente puede seguir esa misma estructura; migrarlos a Repository y Unit of Work es opcional. El árbol muestra el caso en que todos los módulos se ubican en `app/modules/`.

```
proyecto/
├── app/
│   ├── core/
│   │   ├── config.py
│   │   ├── database.py
│   │   ├── repository.py
│   │   └── unit_of_work.py
│   │
│   ├── modules/
│   │   ├── health/
│   │   │   └── router.py
│   │   │
│   │   ├── categorias/
│   │   ├── productos/
│   │   ├── ingredientes/
│   │   └── pedidos/
│   │       ├── models.py
│   │       ├── repository.py
│   │       ├── router.py
│   │       ├── schemas.py
│   │       ├── service.py
│   │       └── unit_of_work.py
│   │
│   └── main.py
├── .env
├── pedidos.http
├── README.md
├── requirements.txt
└── seed.py
```

Ejercicio 2 · Parte B — Frontend con React + TypeScript

65 puntos

Se parte del frontend del TP de la unidad 7, que ya tiene TanStack Query, React Router (con `BrowserRouter` en `main.tsx`) y Tailwind CSS configurados, un `Navbar` con enlaces a `/categorias` y `/productos`, y un `App.tsx` que solo define las rutas. Se conservan las rutas y los nombres de página de esa unidad (`/categorias` → `CategoriasPage` y `/productos` → `ProductsPage`) y se agrega `/ingredientes` → `IngredientesPage`. Se instalan TanStack Form y TanStack Table:

```
pnpm add @tanstack/react-form @tanstack/react-table@8
```

El material de la Actividad 3 usa TanStack Table v8. Desde agosto de 2026 la versión publicada por defecto en npm es la v9, que cambia la API; por eso la instalación indica la versión 8.

a) Páginas

La aplicación debe estar organizada en páginas separadas.

En `src/pages/`:

- `CategoriasPage.tsx` (existe desde la unidad 7; se rehace con TanStack Form y TanStack Table)
- `ProductsPage.tsx` (en la unidad 7 es una página informativa; pasa a implementar el CRUD de productos)
- `IngredientesPage.tsx` (nueva)

Las interfaces `Categoria`, `Producto` e `Ingrediente` se definen en `src/types/`, con los mismos campos que los schemas del backend.

CategoriasPage

Debe contener:

- Toda la lógica de categorías:
  - `useQuery`
  - `useMutation`
  - manejo del modal
  - TanStack Form
  - TanStack Table
- Renderiza:
  - `CategoriaList`
  - `CategoriaModal`
  - botón "Nueva categoría"

ProductsPage

- Estructura similar a `CategoriasPage`
- Implementa el CRUD completo de productos
- Puede reutilizar lógica (recomendado)
- Debe tener:
  - listado
  - `useQuery`
  - `useMutation`
  - manejo del modal
  - TanStack Form
  - TanStack Table

IngredientesPage

- Estructura similar a `CategoriasPage`
- Implementa el CRUD completo de ingredientes
- Puede reutilizar lógica (recomendado)
- Debe tener:
  - listado
  - `useQuery`
  - `useMutation`
  - manejo del modal
  - TanStack Form
  - TanStack Table

b) Navegación

React Router ya está instalado y configurado desde el TP de la unidad 7, con el paquete `react-router-dom` (la guía oficial actual instala `react-router`; `react-router-dom` se mantiene por compatibilidad). En esta unidad solo se agrega lo nuevo:

- En `App.tsx`, la ruta `/ingredientes` → `IngredientesPage`. Las rutas `/`, `/categorias` y `/productos` se mantienen; `/productos` pasa a mostrar el CRUD de productos.
- En `Navbar.tsx`, un `Link` a **Ingredientes**, junto a los de Categorías y Productos.

c) App.tsx

`App.tsx` mantiene el criterio del TP de la unidad 7: no contiene estado de datos ni llamadas a la API, y solo se encarga de:

- Definir rutas
- Renderizar el `Navbar`
- Mostrar las páginas

Comportamiento esperado

- La app inicia en `/` y muestra `CategoriasPage`, como en la unidad 7
- Se puede navegar a `/categorias`, `/productos` e `/ingredientes`
- Cada página maneja su propia lógica
- `App.tsx` no contiene estado de datos ni llamadas a la API

Historias de usuario

Las siguientes historias de usuario describen, desde la perspectiva del actor que opera el sistema, el comportamiento esperado de la aplicación. Cada historia sigue el formato "Como... quiero... para..." y se acompaña de sus criterios de aceptación, que constituyen la condición verificable de que la historia fue implementada correctamente.

HU-01. Listar categorías con TanStack Table

Canal de verificación: Frontend (React).

Como administrador del catálogo, quiero ver todas las categorías en una tabla interactiva con ordenamiento y paginación, para administrar el catálogo.

Criterios de aceptación:

- `CategoriasPage` usa `useQuery` para `GET /categorias` y renderiza con TanStack Table.
- La tabla soporta ordenamiento por columnas (nombre, descripción) con `getSortedRowModel`.
- La tabla pagina los resultados con `getPaginationRowModel` y botones para pasar a la página anterior y a la siguiente (Actividad 3).
- Mientras se cargan los datos se muestra un estado de carga.
- Si no hay categorías, se muestra un estado vacío.
- Los errores de red se informan al usuario.
- Los estados de carga, vacío y error se implementan como componentes reutilizables con Tailwind CSS.

HU-02. Crear categoría con TanStack Form

Canal de verificación: Frontend (React).

Como administrador, quiero crear una categoría usando un formulario con validación tipada, para incorporar categorías nuevas al catálogo con datos válidos.

Criterios de aceptación:

- El botón "Nueva categoría" abre un modal con TanStack Form y validación.
- El formulario muestra los errores de validación de cada campo.
- `useMutation` ejecuta `POST /categorias`; al éxito invalida la query y cierra el modal.

- Si la API responde con un error, el mensaje se muestra sin cerrar el modal (por ejemplo, 409 por nombre duplicado, si se implementa la regla opcional RN-02).

HU-03. Editar categoría

Canal de verificación: Frontend (React).

Como administrador, quiero editar una categoría desde la tabla, para corregir o actualizar sus datos.

Criterios de aceptación:

- El botón Editar abre el modal precargado con TanStack Form.
- `useMutation` ejecuta `PUT /categorias/{id}`; al éxito invalida la caché.
- El modal diferencia alta de edición en título y comportamiento.

HU-04. Eliminar categoría

Canal de verificación: Frontend (React).

Como administrador, quiero eliminar categorías obsoletas, para que el catálogo muestre solo las vigentes.

Criterios de aceptación:

- El botón Eliminar pide confirmación y ejecuta `useMutation` con `DELETE`.
- Si la API responde con un error, se informa al usuario (por ejemplo, 409 si la categoría tiene productos, si se implementa la regla opcional RN-05).
- Tras el éxito, la tabla se actualiza sin recargar.

HU-05. CRUD de Ingredientes

Canal de verificación: Frontend (React) y Backend (REST Client).

Como administrador, quiero gestionar ingredientes con la misma interfaz que categorías, para mantener actualizada la lista de ingredientes.

Criterios de aceptación:

- `IngredientesPage` tiene la misma estructura: `useQuery`, TanStack Table, TanStack Form, `useMutation`.
- `GET /ingredientes`, `POST`, `PUT /{id}`, `DELETE /{id}` con validación de Pydantic.
- El nombre de ingrediente es obligatorio.
- Navegación a `/ingredientes` desde el Navbar.

HU-06. CRUD de Productos

Canal de verificación: Frontend (React) y Backend (REST Client).

Como administrador, quiero gestionar productos con la misma interfaz, para mantener actualizados sus datos, su precio y su disponibilidad.

Criterios de aceptación:

- `ProductsPage` con `useQuery`, TanStack Table, TanStack Form, `useMutation`.
- `GET /productos`, `POST`, `PUT /{id}`, `DELETE /{id}` con validación de Pydantic.
- Productos incluyen nombre, descripción, `precio_base` y `disponible`; el formulario no admite un precio negativo.

HU-07. CRUD de Pedidos y DetallePedido (REST Client)

Canal de verificación: Backend (REST Client).

Como administrador, quiero gestionar pedidos desde el backend, para registrar las ventas y seguir su estado.

Criterios de aceptación:

- `GET /pedidos` lista pedidos; `GET /pedidos/{id}` devuelve el pedido con sus detalles.
- `POST /pedidos` crea el pedido y sus detalles en una sola transacción mediante el Unit of Work (201); si los datos son inválidos (por ejemplo, un producto repetido) responde 422 y, si un detalle referencia un producto inexistente, 404, sin guardar nada.
- `PUT /pedidos/{id}` actualiza el estado (PENDIENTE, CONFIRMADO, ENTREGADO o CANCELADO).
- `DELETE /pedidos/{id}` realiza un borrado lógico.
- Todas las operaciones se demuestran con archivos `.http`.

Reglas de negocio

Las reglas de negocio definen las restricciones que el sistema debe hacer cumplir para mantener la integridad de los datos. Son responsabilidad del backend (capa de validación con Pydantic y lógica de los servicios) y complementan las historias de usuario: una historia describe qué quiere lograr el usuario, mientras que una regla de negocio establece qué está permitido y qué debe rechazarse.

ID	Regla de negocio	Entidad / Verificación
RN-01	Nombre de categoría obligatorio (no vacío ni compuesto solo por espacios).	Categoría / 422
RN-02	Nombre de categoría único (sin distinción de mayúsculas). Opcional.	Categoría / 409
RN-03	Descripción de categoría opcional, default `""`.	Categoría / 201 o 200

ID	Regla de negocio	Entidad / Verificación
RN-04	No operar sobre categoría inexistente.	Categoría / 404
RN-05	No eliminar categoría con productos asociados. Opcional.	Categoría / 409
RN-06	Nombre de producto obligatorio, no único.	Producto / 422
RN-07	`precio_base` >= 0, no negativos.	Producto / 422
RN-08	`disponible` es booleano, default `True`.	Producto / 201
RN-09	No operar sobre producto inexistente.	Producto / 404
RN-10	Nombre de ingrediente obligatorio (no vacío ni compuesto solo por espacios). La unicidad (409) es opcional.	Ingrediente / 422 (409 opcional)
RN-11	No operar sobre ingrediente inexistente.	Ingrediente / 404
RN-12	Un pedido debe tener al menos un DetallePedido para ser creado, y un mismo producto no se repite entre sus detalles.	Pedido / 422
RN-13	No operar sobre pedido inexistente o eliminado.	Pedido / 404
RN-14	El id de cada entidad es autoincremental, asignado por el backend.	Todas / id de solo lectura
RN-15	Los datos se persisten en la base de datos (PostgreSQL con SQLModel). El alta de un pedido se ejecuta dentro de un Unit of Work: si una operación falla, se revierte la transacción completa.	Todas / persistencia tras reiniciar
RN-16	Toda petición con datos inválidos o conflicto no modifica el estado y devuelve error.	Todas / 404, 409 o 422

En RN-01 y RN-10, `min_length=1` por sí solo acepta un nombre compuesto solo por espacios: se eliminan los espacios de los extremos y debe quedar al menos un carácter, por ejemplo con `Annotated[str, StringConstraints(strip_whitespace=True, min_length=1)]` en Pydantic.

Opcional

Las siguientes ampliaciones no son obligatorias ni forman parte de la rúbrica. Permiten practicar con más casos la arquitectura de la Actividad 1 y las funciones de TanStack Form y TanStack Table:

- Implementar el módulo de Ingredientes con Repository y Unit of Work, con nombre único sin distinción de mayúsculas (409, RN-10).
- Asociar ingredientes a los productos (relación Producto–Ingrediente con una tabla de enlace) y permitir seleccionarlos en el formulario de productos.

- Responder 409 en Categoría por nombre duplicado (RN-02) y al eliminar una categoría con productos asociados (RN-05).
- Migrar los módulos de Categoría y Producto del TP de la unidad 7 a la arquitectura Models-Schemas-Repository-UoW-Service-Router.
- Buscar por nombre con un filtro de TanStack Table (`getFilteredRowModel` , con el estado `globalFilter` o `columnFilters`).
- Deshabilitar el botón de guardar mientras el formulario es inválido o se está enviando, con `form.Subscribe` y los valores `canSubmit` e `isSubmitting` (Actividad 2).
- Impedir el cambio de estado de un pedido ENTREGADO o CANCELADO (estados terminales, 409).
- Cargar datos de prueba con `seed.py` .

Conclusiones esperadas

Al finalizar el trabajo práctico, el estudiante debe demostrar:

- **Integración Full Stack:** el frontend consume la API FastAPI con TanStack Query (las funciones de acceso usan `fetch`). Los CRUD de Categorías, Productos e Ingredientes funcionan de extremo a extremo.
- **Modal con useState:** El formulario de alta y edición está dentro de un modal controlado con estado booleano.
- **Hooks aplicados correctamente:** cada página (`CategoriasPage` , `ProductsPage` , `IngredientesPage`) usa `useQuery` y `useMutation` para los datos, `useForm` de TanStack Form para el formulario y `useReactTable` para la tabla. `App.tsx` solo define las rutas y renderiza el `Navbar` . Los componentes de presentación reciben datos y funciones por props.
- **Tipado TypeScript:** Interfaces `Categoria` , `Producto` e `Ingrediente` definidas. Props tipados en todos los componentes `.tsx` .
- **Backend REST documentado:** Endpoints de Pedido y DetallePedido funcionando y demostrados en el archivo `.http` .
- **Arquitectura del backend:** el módulo de pedidos sigue Models-Schemas-Repository-UoW-Service-Router y la creación de un pedido con sus detalles es atómica.
- **CORS configurado:** El backend permite peticiones desde el origen del frontend.

Criterios de evaluación

Criterio	Descripción	Puntos
CRUD Categorías – Frontend	Listar, crear, editar y eliminar categorías desde React usando `useQuery` , `useMutation` , TanStack Form y TanStack Table.	10

Criterio	Descripción	Puntos
CRUD Ingredientes – Frontend	Listar, crear, editar y eliminar ingredientes desde React usando `useQuery`, `useMutation`, TanStack Form y TanStack Table.	10
CRUD Productos – Frontend	Listar, crear, editar y eliminar productos desde React usando `useQuery`, `useMutation`, TanStack Form y TanStack Table.	10
Modal funcional	Modal que se abre para crear y editar categorías, productos e ingredientes, controlado con `useState`. Se cierra al guardar o cancelar.	15
CRUD Pedidos – Backend	Endpoints FastAPI de Pedido y DetallePedido (`GET /pedidos`, `GET /pedidos/{id}`, `POST`, `PUT /{id}`, `DELETE /{id}`) con validación de Pydantic, persistencia en base de datos y arquitectura Models-Schemas-Repository-UoW-Service-Router; la creación del pedido con sus detalles es atómica.	20
CRUD Pedidos y relación – REST Client	Endpoints de Pedido y DetallePedido demostrados con REST Client (`.http`), incluidos los casos 404 y 422. No requiere UI.	15
Tipado TypeScript	Interfaces `Categoria`, `Producto` e `Ingrediente` tipadas. Props tipados en todos los componentes.	10
Estructura y buenas prácticas	Carpetas `components/`, `pages/` y `types/`. Rutas con React Router. Tailwind CSS aplicado, con estados de carga, vacío y error. Sin errores de TypeScript al compilar.	10

Formato de entrega

- Proyecto Frontend comprimido en un archivo `.zip`, sin la carpeta `node_modules`.
- Proyecto Backend comprimido en un archivo `.zip`, sin las carpetas `.venv` y `__pycache__`, con los archivos `.http` de REST Client.

Recursos adicionales

Documentación oficial:

- React: https://react.dev/
- TypeScript: https://www.typescriptlang.org/docs/
- Tailwind CSS: https://tailwindcss.com/docs/
- Vite: https://vite.dev/
- FastAPI: https://fastapi.tiangolo.com/
- SQLModel: https://sqlmodel.tiangolo.com/
- TanStack Query: https://tanstack.com/query/latest

- TanStack Form: https://tanstack.com/form/latest
- TanStack Table (v8): https://tanstack.com/table/v8/docs/introduction
- React Router: https://reactrouter.com/